

CURSO

TEST AUTOMATION ENGINEER

FORMACIÓN INTEGRAL

Petstore

→ Get the list of all the pets

GET http://127.0.0.1:8200/api/pet [200 OK, 699B, 127ms]

- ✓ Payload must be valid and have a body
- ✓ Content-Type is present
- ✓ Response time is less than 200ms

```
[ { id: 1,
  category: { id: 1, name: 'labrador re
triever' },
  name: 'doggie',
  photoUrls: [ '/images/doggie.png' ],
  tags:
    [ { id: 1, name: 'labrador' },
      { id: 2, name: 'retriever' },
      { id: 3, name: 'dog' } ],
  status: 'available' },
  { id: 2,
    category: { id: 2, name: 'angora turc
o' },
    name: 'catty',
    photoUrls: [ '/images/catty.png' ],
    tags:
      [ { id: 0, name: 'angora' }, { id
9m: 0, name: 'turco' }, { id: 0,
7mname: 'cat' } ],
    status: 'available' } ]
```

Objetivo General del Curso

ANALIZAR LAS HERRAMIENTAS DE DISEÑO DE TEST AUTOMATION ENGINEER, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Objetivo específico del Módulo

EMPLEAR PRUEBAS DE APIS CON POSTMAN Y SUPertest,
DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Contenidos

Objetivo General del Curso	2
Objetivo específico del Módulo.....	2
Módulo 6, PRUEBAS DE APIS CON POSTMAN Y SUPERTEST	4
Capítulo 1: Fundamentos de REST APIS Y JSON	6
¿Qué es una API?	6
¿Qué es una API REST?	7
¿Qué es JSON?	10
Capítulo 2: POSTMAN: Colección, validaciones y ambiente de pruebas	13
Postman: Colección, Validaciones y Ambiente de Pruebas	13
Capítulo 3: Automatización con SUPERTEST + JEST	17
¿Qué es Supertest?	17
¿Qué es Jest?	18
Estructura típica de pruebas con Supertest y Jest	18
Buenas prácticas en pruebas con Supertest y Jest	23
Cobertura de pruebas	23
Integración con bases de datos	24
Pruebas en entornos CI/CD	24
Capítulo 4: PRUEBAS AUTENTICADAS Y VALIDACIÓN DE ERRORES	25
Pruebas Autenticadas	25
Validación de Errores.....	27
Estandarización de errores.....	29
Capítulo 5: INTEGRACIÓN CON PRUEBAS E2E.....	31
¿Por qué integrar APIs en pruebas E2E?	31
¿Cómo realizar pruebas E2E con APIs?	32
Integración con Front-End	34
Beneficios de integrar pruebas E2E con APIs	35
Buenas prácticas	35

Módulo 6, PRUEBAS DE APIS CON POSTMAN Y SUPERTTEST



Las pruebas de APIs son una parte esencial del desarrollo moderno de software, ya que permiten verificar que los servicios web funcionen correctamente, respondan de manera oportuna y entreguen datos válidos de acuerdo con los requerimientos del sistema. En un contexto donde las aplicaciones dependen cada vez más de la integración con servicios externos o arquitecturas distribuidas basadas en microservicios, asegurar la calidad y el correcto funcionamiento de las APIs se vuelve crucial para garantizar la estabilidad del producto final, la satisfacción del usuario y la escalabilidad de la solución.

Este capítulo tiene como objetivo introducir y guiar al lector en la realización de pruebas efectivas sobre APIs REST, utilizando dos herramientas ampliamente adoptadas por la industria: Postman, ideal para pruebas manuales, exploratorias y colaborativas; y Supertest, una biblioteca poderosa que permite automatizar pruebas de APIs en entornos de desarrollo Node.js, integrándose fácilmente con frameworks de testing como Mocha o Jest. Ambas herramientas cumplen roles complementarios en el ciclo de vida del desarrollo y la validación de software.

Sin embargo, antes de profundizar en el uso práctico de estas herramientas, es fundamental comprender los principios sobre los cuales están construidas las APIs REST, así como el formato estándar de intercambio de información que utilizan: JSON (JavaScript Object Notation). Estos conocimientos proporcionan el contexto técnico necesario para interpretar adecuadamente las respuestas, estructurar solicitudes correctas y diseñar casos de prueba sólidos que reflejen el comportamiento esperado de la API en distintos escenarios.



Capítulo 1: FUNDAMENTOS DE REST APIS Y JSON

¿Qué es una API?

Una API (Interfaz de Programación de Aplicaciones, por sus siglas en inglés) es un conjunto estructurado de reglas, definiciones y protocolos que permite la comunicación entre distintos sistemas o componentes de software. Su principal función es actuar como intermediario entre dos aplicaciones que necesitan intercambiar datos o ejecutar operaciones de forma estandarizada y segura, sin necesidad de que una conozca los detalles internos del funcionamiento de la otra.

En el contexto del desarrollo web moderno, una API cumple un rol fundamental al facilitar la interacción entre el cliente (como una aplicación móvil, una aplicación de escritorio o un navegador web) y el servidor que aloja la lógica del negocio y los datos. A través de esta interfaz, el cliente puede realizar diversas operaciones, como consultar información (lectura), enviar nuevos datos (creación), modificar registros existentes (actualización) o eliminar elementos (borrado). Estas acciones se realizan normalmente a través de solicitudes HTTP, que el servidor procesa y responde, permitiendo así un flujo de datos estructurado y consistente entre las partes.

Gracias a las APIs, los desarrolladores pueden integrar funcionalidades complejas de manera eficiente, como pasarelas de pago, servicios de autenticación, geolocalización, almacenamiento en la nube, redes sociales o motores de búsqueda, sin tener que desarrollar esas funciones desde cero. Además, las APIs fomentan la modularidad, escalabilidad y reutilización del código, lo que facilita el mantenimiento y evolución de los sistemas a lo largo del tiempo.

En resumen, una API no solo conecta aplicaciones, sino que establece un lenguaje común para que colaboren eficazmente, independientemente de las tecnologías o plataformas en las que estén construidas.

¿Qué es una API REST?

Una API REST (Representational State Transfer) es un estilo de arquitectura para el diseño de servicios web que permite la comunicación entre sistemas distribuidos utilizando el protocolo HTTP.

Las APIs RESTful se han convertido en el estándar más utilizado en el desarrollo de aplicaciones modernas debido a su simplicidad, flexibilidad, escalabilidad y amplia compatibilidad con la mayoría de los lenguajes de programación, frameworks y plataformas.

REST no es una tecnología en sí misma, sino un conjunto de principios arquitectónicos definidos por Roy Fielding en su tesis doctoral en el año 2000. Estos principios promueven una estructura clara y uniforme para la interacción entre clientes y servidores, lo que facilita tanto la comprensión como el desarrollo e integración de servicios.

A diferencia de otras arquitecturas más complejas, como SOAP, REST permite que los recursos del sistema (como usuarios, productos o publicaciones) se manipulen utilizando métodos estándar del protocolo HTTP, y se identifiquen mediante URIs (Uniform Resource Identifiers).

Esta combinación de reglas simples pero potentes ha permitido a REST ganar una enorme adopción, especialmente en entornos donde se requiere alta interoperabilidad y eficiencia en el consumo de recursos.

A continuación, se detallan las principales características de una API RESTful:

Stateless (Sin estado):

Una API REST debe ser stateless, lo que significa que cada solicitud del cliente al servidor debe contener toda la información necesaria para que el servidor procese la petición. El servidor no debe almacenar ningún estado o contexto de la sesión del cliente entre peticiones.

Esta propiedad mejora la escalabilidad del sistema, ya que no se requiere mantener sesiones abiertas en el servidor.

Uso de métodos HTTP estándar:

REST se basa en el uso de los métodos HTTP tradicionales para definir las operaciones que se pueden realizar sobre los recursos. Cada método tiene un propósito específico:

- **GET:** Recupera datos o información de un recurso existente. Es una operación de solo lectura.
- **POST:** Envía datos al servidor para crear un nuevo recurso.
- **PUT:** Actualiza completamente un recurso existente con los datos enviados.
- **PATCH:** Modifica parcialmente un recurso (es más liviano que PUT).
- **DELETE:** Elimina un recurso del servidor.

Este uso intuitivo del protocolo HTTP facilita la implementación y prueba de las APIs RESTful.

Recursos identificados por URIs:

En REST, los elementos con los que se interactúa (llamados recursos) se identifican mediante URIs únicas. Cada recurso puede representarse por una dirección web específica, lo que permite un acceso directo y claro. Por ejemplo:

```
bash
```

```
GET /usuarios/123
```

Esta URI hace referencia al usuario con ID 123 y permite acceder a sus datos. El uso de URIs bien diseñadas ayuda a mantener una estructura lógica y coherente en la API.

Formato común de respuesta:

Aunque REST permite diferentes formatos de respuesta (como XML, YAML o incluso texto plano), el más utilizado hoy en día es JSON (JavaScript Object Notation). JSON es un formato ligero, fácil de leer tanto por humanos como por máquinas, y se integra de forma natural con la mayoría de los lenguajes de programación.

Una respuesta típica en formato JSON podría verse así:

```
json

{
  "id": 123,
  "nombre": "María",
  "email": "maria@ejemplo.com"
}
```

Este tipo de respuestas estandarizadas facilita el procesamiento, la validación y la visualización de los datos en las aplicaciones cliente.

En resumen, una API REST proporciona una forma organizada, eficiente y universalmente comprensible de construir servicios web que puedan ser consumidos por múltiples tipos de clientes. Su simplicidad y potencia la convierten en una herramienta clave en el desarrollo de sistemas modernos, desde aplicaciones móviles hasta plataformas empresariales basadas en la nube.

¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato ligero y ampliamente utilizado para el intercambio de datos estructurados entre un cliente y un servidor. Diseñado originalmente como una derivación de la sintaxis de objetos de JavaScript, JSON se ha convertido en un estándar universal gracias a su simplicidad, legibilidad y compatibilidad con la mayoría de los lenguajes de programación modernos.

Una de sus principales fortalezas es que está diseñado para ser fácil de leer y escribir por los humanos, al mismo tiempo que resulta rápido de analizar ("parsear") y generar por las máquinas. Esta dualidad lo convierte en un formato ideal para aplicaciones web, móviles, de escritorio y servicios que requieren transmitir datos estructurados de forma eficiente y comprensible.

Estructura básica de JSON

Los datos en JSON se representan utilizando dos estructuras principales:

- **Objetos:** Definidos por llaves {} y compuestos por pares clave-valor. Cada clave debe estar entre comillas dobles.

Ejemplo:

```
json

{
  "nombre": "Carlos",
  "edad": 30,
  "activo": true
}
```

- **Arreglos (Arrays):** Definidos por corchetes [] y compuestos por una lista ordenada de valores.

Ejemplo:

```
json
```

```
[  
  "Chile",  
  "Argentina",  
  "Perú"  
]
```

Los valores en JSON pueden ser de diferentes tipos de datos: cadenas de texto (strings), números, booleanos (true o false), null, objetos u otros arreglos.

JSON y las APIs REST

El uso de JSON como formato de intercambio en las APIs REST se ha vuelto casi omnipresente por varias razones:

- **Ligereza:** A diferencia de otros formatos como XML, JSON es menos verboso y produce archivos más pequeños, lo cual reduce el consumo de ancho de banda y mejora los tiempos de respuesta.
- **Fácil integración:** Prácticamente todos los lenguajes de programación cuentan con bibliotecas o funciones nativas para interpretar (parsear) y generar JSON.
- **Compatibilidad con JavaScript:** Dado que JSON es nativo en JavaScript, puede ser manipulado directamente por navegadores sin necesidad de transformación, lo cual lo hace ideal para aplicaciones frontend.
- **Estandarización:** Al ser tan ampliamente adoptado, su estructura se ha estandarizado, facilitando la interoperabilidad entre distintos sistemas y servicios.

Ejemplo de intercambio con JSON

Cuando un cliente realiza una solicitud a una API REST, el servidor puede responder con un cuerpo en formato JSON como este:

```
json

{
  "id": 456,
  "titulo": "Introducción a las APIs",
  "autor": "Laura Soto",
  "publicado": true
}
```

Del mismo modo, si el cliente desea enviar datos al servidor (por ejemplo, al crear un nuevo usuario), puede incluir un cuerpo JSON en la solicitud:

```
json

{
  "nombre": "Ana",
  "correo": "ana@ejemplo.com",
  "rol": "administradora"
}
```

Esta simplicidad en la estructura facilita tanto la lectura como la validación de los datos transmitidos.

En conclusión, JSON ha revolucionado la manera en que las aplicaciones modernas manejan e intercambian información, convirtiéndose en un pilar esencial para el funcionamiento de APIs REST. Su equilibrio entre legibilidad humana y eficiencia técnica lo posiciona como una herramienta indispensable en cualquier entorno de desarrollo web o móvil.

Capítulo 2: POSTMAN: COLECCIÓN, VALIDACIONES Y AMBIENTE DE PRUEBAS

Postman: Colección, Validaciones y Ambiente de Pruebas

Postman es una herramienta gráfica muy popular utilizada por desarrolladores, testers y equipos de QA para diseñar, probar, documentar y automatizar APIs. Su interfaz intuitiva y su robusta funcionalidad permiten realizar pruebas de forma rápida y eficiente, sin necesidad de escribir código. Es especialmente útil en entornos de desarrollo colaborativo y ciclos ágiles, donde validar continuamente el comportamiento de una API resulta clave.

Colecciones en Postman

Las colecciones son una característica central de Postman. Permiten organizar múltiples peticiones HTTP agrupadas por funcionalidades, versiones o rutas de una API. Cada colección puede contener:

- Peticiones (GET, POST, PUT, DELETE, etc.)
- Variables reutilizables (como tokens de autenticación, URLs base, parámetros de entorno)
- Scripts de prueba y scripts pre-solicitud
- Documentación integrada
- Ejecuciones automatizadas

Ventajas de usar colecciones:

- Permiten mantener orden en el trabajo de pruebas.
- Facilitan la colaboración, ya que se pueden compartir entre miembros del equipo o sincronizar mediante Postman Cloud.
- Se pueden exportar e importar fácilmente entre entornos o proyectos.
- Funcionan como base para pruebas automatizadas con la herramienta Newman (CLI de Postman).

Ejemplo de estructura de colección:

```
bash
```

Colección: API de Usuarios

- └─ Obtener todos los usuarios (GET /usuarios)
- └─ Crear nuevo usuario (POST /usuarios)
- └─ Actualizar usuario por ID (PUT /usuarios/:id)
- └─ Eliminar usuario (DELETE /usuarios/:id)

Validaciones (Tests) en Postman

Postman permite escribir tests automatizados utilizando JavaScript. Estos tests se ejecutan después de recibir la respuesta de una petición, y permiten validar distintos aspectos de la misma, como el código de estado HTTP, la estructura del cuerpo de respuesta, los tiempos de respuesta o la presencia de ciertos encabezados.

Ejemplos comunes de validaciones:

```
javascript
```

```
// Validar que la respuesta devuelva código 200
pm.test("Código de estado es 200", function () {
  pm.response.to.have.status(200);
});

// Validar que el cuerpo de respuesta incluya un campo 'nombre'
pm.test("El campo 'nombre' está presente", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("nombre");
});

// Validar tiempo de respuesta menor a 500ms
pm.test("Tiempo de respuesta menor a 500ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

Estas validaciones ayudan a detectar errores rápidamente y aseguran que la API cumpla con las especificaciones funcionales y de rendimiento.

Ambientes de pruebas en Postman

Un ambiente en Postman es un conjunto de variables que se puede usar para ejecutar peticiones en diferentes contextos (por ejemplo, desarrollo, staging o producción), sin necesidad de modificar manualmente cada URL o token en las solicitudes.

Ventajas de los ambientes:

- Permiten parametrizar URLs base, tokens de autenticación, credenciales y otros datos sensibles o cambiantes.
- Facilitan la reutilización y mantenimiento de las colecciones.
- Ayudan a simular múltiples escenarios de ejecución de forma rápida y segura.

Ejemplo de variables de ambiente:

```
json

{
  "base_url": "https://api.miservicio.com",
  "token": "1234abcd5678efgh"
}
```

Una vez definido el ambiente, se puede usar en las peticiones de la siguiente forma:

```
bash

{{base_url}}/usuarios
```

Esto mejora la portabilidad del proyecto y reduce los errores derivados de cambios manuales en múltiples partes del entorno.

Automatización y documentación

Además de pruebas manuales, Postman permite ejecutar colecciones de forma automática usando herramientas como Newman, útil para integrar las pruebas en pipelines de integración continua (CI/CD). También ofrece capacidades de generación automática de documentación interactiva, lo que facilita la comunicación con otros equipos o clientes.

En resumen, Postman es una herramienta integral que no solo facilita las pruebas manuales de APIs, sino que también permite establecer un flujo de trabajo robusto y automatizado mediante el uso de colecciones, validaciones y ambientes.

Dominar estas funcionalidades permite asegurar la calidad del software, detectar errores antes de pasar a producción y mejorar la colaboración entre los equipos de desarrollo.

Capítulo 3: AUTOMATIZACIÓN CON SUPertest + JEST

En el desarrollo moderno de aplicaciones backend, especialmente aquellas construidas con Node.js y Express, la automatización de pruebas es fundamental para garantizar la estabilidad, la confiabilidad y la evolución segura del software. Las pruebas automatizadas permiten verificar continuamente que los endpoints de una API funcionen correctamente, incluso cuando se realizan cambios en el código base. En este contexto, dos herramientas ampliamente utilizadas son Supertest y Jest.

Supertest es una librería ligera que permite realizar pruebas HTTP sobre servidores Node.js como si fueran clientes reales, mientras que Jest es un framework completo de pruebas desarrollado por Meta (Facebook) que incluye ejecución de pruebas, aserciones, mocks, cobertura de código y funcionalidades avanzadas para testing de aplicaciones JavaScript.

La combinación de ambas herramientas permite escribir pruebas rápidas, robustas y mantenibles para APIs RESTful, con una integración fluida en flujos de integración continua (CI/CD).

¿Qué es Supertest?

Supertest es una herramienta diseñada para testear HTTP endpoints. Internamente usa la librería `superagent` para enviar peticiones y está pensada para probar aplicaciones basadas en Express.js, aunque también se puede utilizar con otros frameworks.

Su principal ventaja es que permite hacer pruebas sobre una instancia de servidor sin necesidad de levantar un servidor real (no requiere estar escuchando en un puerto), lo que la hace ideal para pruebas unitarias y de integración.

Ventajas:

- Simula peticiones HTTP reales a tus rutas Express.
- Compatible con cualquier framework de pruebas (Jest, Mocha, etc.).
- Fácil de usar e integrar en entornos automatizados.
- Permite realizar pruebas de autenticación, middleware, controladores, validaciones, etc.

¿Qué es Jest?

Jest es un framework de testing completo para aplicaciones JavaScript. Es ampliamente usado por su facilidad de configuración, rapidez de ejecución, herramientas integradas y su enfoque en pruebas predictivas y consistentes.

Características:

- Zero configuration (sin necesidad de configuración inicial compleja).
- Compatible con pruebas síncronas y asíncronas.
- Cobertura de código incorporada (--coverage).
- Excelente sistema de mocking para simular dependencias.
- Integración con Babel, TypeScript, y otras herramientas modernas.

Estructura típica de pruebas con Supertest y Jest

Una prueba típica de API con Supertest y Jest consiste en:

- Crear una instancia de la aplicación (por ejemplo, tu app Express).
- Usar Supertest para hacer una petición a un endpoint.
- Evaluar la respuesta (status code, cuerpo, headers, etc.).
- Usar Jest para hacer las aserciones correspondientes.

Instalación

Primero, debe instalar ambas herramientas en su proyecto Node.js:

```
bash

npm install --save-dev supertest jest
```

También es recomendable tener nodemon y cross-env si trabaja en entornos de desarrollo y pruebas más avanzadas.

En package.json, agregue un script para correr las pruebas:

```
json

"scripts": {
  "test": "jest"
}
```

Ejemplo básico de prueba con Supertest + Jest

Archivo: app.js

```
javascript

const express = require('express');
const app = express();

app.use(express.json());

app.get('/saludo', (req, res) => {
  res.status(200).json({ mensaje: 'Hola Mundo' });
});

module.exports = app;
```

Archivo: app.test.js

```
javascript

const request = require('supertest');
const app = require('./app');

describe('GET /saludo', () => {
  it('debería responder con un saludo', async () => {
    const response = await request(app).get('/saludo');
    expect(response.statusCode).toBe(200);
    expect(response.body.mensaje).toBe('Hola Mundo');
  });
});
```

Pruebas con métodos HTTP variados

Supertest permite trabajar fácilmente con los métodos GET, POST, PUT, PATCH y DELETE. Un ejemplo con POST podría ser:

javascript

```
describe('POST /usuarios', () => {
  it('debería crear un nuevo usuario', async () => {
    const nuevoUsuario = { nombre: 'Ana', email: 'ana@example.com' };

    const res = await request(app)
      .post('/usuarios')
      .send(nuevoUsuario)
      .set('Accept', 'application/json');

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body.nombre).toBe('Ana');
  });
});
```

Pruebas de autenticación y headers

Puede probar endpoints protegidos por tokens o cabeceras personalizadas:

javascript

```
it('debería rechazar acceso sin token', async () => {
  const res = await request(app).get('/perfil');
  expect(res.statusCode).toBe(401);
});

it('debería permitir acceso con token válido', async () => {
  const token = 'Bearer eyJhbGciOi...';
  const res = await request(app)
    .get('/perfil')
    .set('Authorization', token);
  expect(res.statusCode).toBe(200);
});
```

Buenas prácticas en pruebas con Supertest y Jest

- Separar pruebas unitarias de integración.
- Usar `beforeEach` y `afterEach` para limpiar datos entre pruebas (útil con bases de datos).
- Mantener datos de prueba predecibles y consistentes.
- Evitar dependencias externas reales en pruebas (mockear si es necesario).
- Cubrir escenarios positivos y negativos (estatus 200, 400, 404, 500).
- Incluir validaciones de headers, tiempos de respuesta y formatos.

Cobertura de pruebas

Con Jest, puedes medir la cobertura de tus pruebas fácilmente con:

```
bash  
  
npx jest --coverage
```

Esto le mostrará:

- Qué funciones, líneas y ramas de tu código han sido testeadas.
- Qué archivos requieren mayor atención.
- Cuánto de tu código productivo está cubierto por pruebas.

Integración con bases de datos

Para pruebas que involucran persistencia de datos (por ejemplo, MongoDB, PostgreSQL):

- Use bases de datos de prueba separadas del entorno de producción.
- Puede usar librerías como mongodb-memory-server o fixtures en SQLite.
- Asegúrese de limpiar la base entre pruebas para evitar efectos colaterales.

Pruebas en entornos CI/CD

Supertest + Jest se integran perfectamente en pipelines de CI/CD como GitHub Actions, GitLab CI, Jenkins, etc.

Puede correr las pruebas como parte de su flujo de integración continua para asegurar que ningún cambio rompa funcionalidades existentes.

La combinación de Supertest y Jest representa una solución poderosa, elegante y eficiente para la automatización de pruebas en APIs construidas con Node.js. Mientras Supertest proporciona una interfaz clara para testear endpoints HTTP como si se tratara de clientes reales, Jest ofrece un entorno de ejecución de pruebas robusto, rápido y lleno de funcionalidades modernas. Juntas, estas herramientas permiten implementar estrategias de pruebas continuas (continuous testing), detectar errores en etapas tempranas, y mantener un alto nivel de calidad en el desarrollo de servicios backend escalables y confiables.

Capítulo 4: PRUEBAS AUTENTICADAS Y VALIDACIÓN DE ERRORES

En el desarrollo de APIs REST, uno de los aspectos más críticos es asegurar que los mecanismos de autenticación funcionen correctamente y que los errores se manejen de manera consistente, segura y comprensible. Las pruebas autenticadas permiten verificar que los usuarios accedan únicamente a los recursos para los que están autorizados, mientras que la validación de errores garantiza que la API responda de manera predecible y clara frente a situaciones anómalas o solicitudes incorrectas.

La ausencia de pruebas en estas áreas puede derivar en graves fallos de seguridad, fuga de información sensible, mal funcionamiento de la lógica de negocio y una pobre experiencia de usuario.

Pruebas Autenticadas

Las pruebas autenticadas simulan el comportamiento de usuarios que acceden a la API protegida mediante mecanismos como tokens JWT, OAuth2, API Keys u otros esquemas. El objetivo es verificar que:

- Los endpoints protegidos requieren credenciales válidas.
- Las credenciales inválidas o ausentes sean rechazadas con los códigos de estado apropiados (ej. 401 Unauthorized, 403 Forbidden).
- El acceso se controle de acuerdo al rol o nivel de permiso del usuario (administrador, usuario básico, invitado, etc.).
- Los tokens expirados o manipulados no sean aceptados.

Ejemplo con JWT y Supertest:

```
javascript

describe('GET /perfil - acceso autenticado', () => {
  let token;

  beforeEach(async () => {
    // Simulación de login para obtener el token
    const res = await request(app)
      .post('/auth/login')
      .send({ email: 'usuario@example.com', password: '123456' });

    token = res.body.token;
  });

  it('debería permitir el acceso con token válido', async () => {
    const res = await request(app)
      .get('/perfil')
      .set('Authorization', `Bearer ${token}`);

    expect(res.statusCode).toBe(200);
    expect(res.body).toHaveProperty('nombre');
  });

  it('debería rechazar acceso sin token', async () => {
    const res = await request(app).get('/perfil');
    expect(res.statusCode).toBe(401);
  });

  it('debería rechazar acceso con token inválido', async () => {
    const res = await request(app)
      .get('/perfil')
      .set('Authorization', 'Bearer token-falso');

    expect(res.statusCode).toBe(401);
  });
});
```

Validación de Errores

Una API bien diseñada no solo responde correctamente ante solicitudes válidas, sino que también gestiona con precisión y claridad las solicitudes erróneas.

La validación de errores debe contemplar:

- Errores de validación de entrada: campos faltantes, formatos incorrectos, restricciones no cumplidas.
- Errores de lógica de negocio: usuario no encontrado, saldo insuficiente, etc.
- Errores de autenticación/autorización.
- Errores del servidor (500): excepciones no controladas.
- Recursos no encontrados (404).

El uso de respuestas estandarizadas permite a los clientes interpretar fácilmente qué salió mal y cómo corregirlo.

Estas respuestas suelen incluir:

- Código de estado HTTP adecuado.
- Mensaje de error legible.
- Código interno de error (opcional).
- Detalles adicionales o sugerencias de corrección.

Ejemplo de validación de campos en una petición POST:

```
javascript

describe('POST /usuarios - validación de entrada', () => {
  it('debería fallar si falta el email', async () => {
    const res = await request(app)
      .post('/usuarios')
      .send({ nombre: 'Laura' });

    expect(res.statusCode).toBe(400);
    expect(res.body.error).toBe('El campo email es requerido.');
```

```
  });
  it('debería fallar si el email es inválido', async () => {
    const res = await request(app)
      .post('/usuarios')
      .send({ nombre: 'Laura', email: 'no-es-un-email' });

    expect(res.statusCode).toBe(400);
    expect(res.body.error).toMatch(/email.*inválido/i);
  });
});
```

Pruebas de límites y valores extremos

Es importante testear no solo los flujos esperados, sino también aquellos que podrían vulnerar o saturar el sistema:

- Cadenas de texto extremadamente largas.
- Peticiones con headers malformados.
- Reenvío de la misma solicitud varias veces.
- Tokens con firma válida pero payload manipulado.

Este tipo de pruebas permite descubrir vulnerabilidades, prevenir ataques de denegación de servicio y asegurar la robustez del backend.

Estandarización de errores

Una API profesional debería definir un formato uniforme para los errores. Un ejemplo común es:

```
json

{
  "status": 400,
  "error": "Bad Request",
  "message": "El campo 'email' es requerido.",
  "timestamp": "2025-04-20T13:00:00Z"
}
```

Este formato puede ser validado directamente en las pruebas automatizadas:

```
javascript

expect(res.body).toEqual(
  expect.objectContaining({
    status: 400,
    error: 'Bad Request',
    message: expect.any(String),
    timestamp: expect.any(String),
  })
);
```

Pruebas de autorización por rol

Si la API implementa control de acceso por roles, es esencial verificar que los privilegios estén correctamente asignados:

javascript

```
it('no debería permitir eliminar usuarios a un rol no administrador', async () => {  
  const tokenUsuario = await obtenerToken('usuario@example.com', '1234');  
  
  const res = await request(app)  
    .delete('/usuarios/5')  
    .set('Authorization', `Bearer ${tokenUsuario}`);  
  
  expect(res.statusCode).toBe(403);  
  expect(res.body.message).toBe('No tienes permisos para realizar esta acción.');
```

Las pruebas autenticadas y la validación de errores son componentes imprescindibles dentro de una suite de pruebas bien diseñada. A través de estas pruebas, es posible garantizar que la seguridad, la experiencia del usuario y la integridad de los datos no se vean comprometidas. Incorporar este tipo de validaciones como parte del proceso de desarrollo continuo permite detectar vulnerabilidades tempranamente, mantener la coherencia en las respuestas, y asegurar que la API esté alineada con las mejores prácticas del desarrollo de software moderno.

Capítulo 5: INTEGRACIÓN CON PRUEBAS E2E

Las pruebas End-to-End (E2E) representan el nivel más alto en la jerarquía de pruebas de software. Su propósito principal es verificar que todo el sistema funcione correctamente desde la perspectiva del usuario final, simulando el recorrido completo de un flujo funcional —desde la capa de presentación hasta la base de datos, pasando por los servicios intermedios, incluidas las APIs. Cuando se integran pruebas de API dentro del contexto E2E, se asegura que todos los componentes que dependen de las APIs (front-end, servicios externos, procesos de negocio) interactúan correctamente en condiciones reales.

¿Por qué integrar APIs en pruebas E2E?

Una API puede pasar todas sus pruebas unitarias y de integración, pero aún fallar cuando se utiliza dentro de un flujo completo. Por ejemplo, una API de autenticación puede emitir un token correctamente, pero si el front-end no puede consumirlo debido a un error de formato o expiración anticipada, el sistema se rompe para el usuario.

Integrar pruebas de API en E2E permite:

- Validar que el front-end pueda consumir correctamente los endpoints.
- Asegurar que los flujos completos del sistema respondan según lo esperado.
- Detectar errores de integración entre módulos (por ejemplo, entre microservicios o servicios externos).
- Verificar flujos críticos como registro, login, compra, publicación de contenido, gestión de usuarios, etc.

¿Cómo realizar pruebas E2E con APIs?

Las pruebas E2E pueden realizarse con herramientas como Cypress, Playwright, Selenium, o bien combinarse con pruebas automatizadas de backend usando Supertest + Jest, junto con una base de datos temporal y servidores de prueba.

Un enfoque común consiste en:

- Inicializar la base de datos en un estado controlado (fixtures, mocks o base temporal).
- Simular interacciones reales: login, creación de recursos, edición, eliminación.
- Verificar que las respuestas de la API se alinean con los cambios en la UI o en otras capas del sistema.
- Validar el estado final del sistema (respuestas HTTP, cambios en la base de datos, existencia de archivos, etc.).

Ejemplo práctico:

FLUJO DE COMPRA

Imagine una plataforma de e-commerce. Un flujo E2E podría validar lo siguiente:

- Un usuario se registra.
- Inicia sesión.
- Agrega productos al carrito.
- Realiza un pedido.
- Recibe una confirmación por correo electrónico o en pantalla.

Prueba automatizada usando Supertest + Jest (resumen simplificado):

```
describe('Flujo E2E: Compra de producto', () => {
  let token;
  let productId;

  beforeAll(async () => {
    // Registro
    await request(app).post('/usuarios').send({
      nombre: 'Carlos',
      email: 'carlos@ejemplo.com',
      password: '123456'
    });

    // Login
    const resLogin = await request(app).post('/auth/login').send({
      email: 'carlos@ejemplo.com',
      password: '123456'
    });

    token = resLogin.body.token;

    // Crear producto
    const resProducto = await request(app)
      .post('/productos')
      .set('Authorization', `Bearer ${token}`)
      .send({
        nombre: 'Auriculares',
        precio: 35000,
        stock: 10
      });

    productId = resProducto.body.id;
  });

  it('debería permitir comprar un producto', async () => {
    const res = await request(app)
      .post('/pedidos')
      .set('Authorization', `Bearer ${token}`)
      .send({
        productos: [{ id: productId, cantidad: 1 }]
      });

    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('pedidoId');
    expect(res.body.estado).toBe('confirmado');
  });
});
```

Integración con Front-End

En pruebas E2E completas, también puede usarse una herramienta como Cypress para automatizar flujos de usuario que involucren interacciones directas con APIs REST desde la interfaz gráfica.

Ejemplo en Cypress:

```
javascript

cy.request('POST', '/api/auth/login', {
  email: 'usuario@ejemplo.com',
  password: '123456'
}).then((response) => {
  expect(response.status).to.eq(200);
  const token = response.body.token;

  cy.visit('/perfil', {
    onBeforeLoad(win) {
      win.localStorage.setItem('token', token);
    }
  });

  cy.contains('Bienvenido');
});
```

Este enfoque permite validar que:

- El cliente puede manejar correctamente los tokens.
- La API está entregando los datos esperados para la UI.
- No hay problemas de CORS, expiración de sesión, o rutas protegidas mal implementadas.

Beneficios de integrar pruebas E2E con APIs

- Cobertura total: valida la experiencia del usuario desde el inicio al fin.
- Pruebas realistas: simulan el comportamiento real del sistema.
- Prevención de regresiones: aseguran que nuevas funcionalidades no rompan las existentes.
- Confianza para despliegues: se convierten en un paso clave en pipelines de CI/CD.

Buenas prácticas

- Usar entornos de pruebas aislados con bases de datos limpias o mockeadas.
- Automatizar los flujos más críticos (login, pagos, creación de usuarios).
- Limitar la cantidad de pruebas E2E pesadas en cada ejecución (priorizar las esenciales).
- Incluir pruebas de API tanto independientes como dentro de los flujos E2E.
- Utilizar herramientas como Docker o test containers para levantar ambientes reproducibles.

La integración de pruebas E2E con APIs REST permite verificar que el sistema funcione de manera coherente, robusta y segura desde la perspectiva del usuario final. Combinadas con herramientas como Supertest, Jest, Cypress o Playwright, estas pruebas se convierten en una defensa clave contra errores de integración, regresiones y fallos en la experiencia del cliente.

Implementar este enfoque fortalece la calidad del software y reduce el riesgo de fallos en producción.